SVKM's NMIMS

School of Technology Management & Engineering (STME)

MINI-PROJECT REPORT

Real-Time Sign Language to Speech Conversion using Hand Landmark Detection and Classical Image Processing

Course: Image and Video Processing

Student Name	Mahima Singh
Roll Number	B014
SAP ID	70472300260
Programme	MBA-Tech (CE)
Semester / Year	5th Semester / 3rd Year
Faculty Name	Dr. Rajgaurav Mishra
Institute	SVKM's NMIMS
Department	STME (School of Technology Management & Engineering)
Academic Year	2026–27
Submission Date	13 October 2026

Abstract

Communication between sign language users and people who do not know sign language remains a barrier in education, healthcare and public services. This project presents a real-time system that converts static American Sign Language (ASL) fingerspelling gestures, captured by an ordinary webcam, into text and synthesised speech. The proposed solution combines MediaPipe hand-landmark detection with a classical image-processing pipeline. Each frame is converted to the HSV colour space, skin pixels are thresholded, the mask is cleaned with morphological opening and closing, and the largest contour is analysed using the convex hull, convexity defects and Hu moments. These 13 shape features are concatenated with 63 normalised landmark coordinates to form a 76-dimensional vector, classified by a Random Forest trained on a self-recorded dataset of 24 ASL letters. A temporal stability filter turns per-frame predictions into reliable letters, which are assembled into words and spoken through an offline text-to-speech engine. The classifier reached 99.69% test accuracy (87.3% with the classical features alone), and the complete system ran interactively on a laptop CPU. Limitations include sensitivity of skin segmentation to lighting, single-signer training data and the inability to represent motion-based signs.

1. Introduction

Background. Sign languages are complete visual languages used by deaf and hard-of-hearing communities, built from hand shapes, movements, facial expressions and body posture. According to the World Health Organization, more than 1.5 billion people live with some degree of hearing loss, and about 430 million have disabling hearing loss [1]. Most hearing people do not know any sign language, and professional interpreters are scarce and costly, so everyday interactions such as a hospital visit, a bank counter or a classroom discussion can become difficult.

Relevance. A low-cost system that translates signs into text and speech in real time can act as a communication bridge. Because it needs only a webcam, it can be deployed on a laptop or a kiosk without special gloves or sensors. Fingerspelling, in which each letter of the alphabet has its own hand shape, is the natural starting point: it is used for names, places and words that have no dedicated sign, and its static nature makes it suitable for image-based recognition.

Role of image and video processing. The input to this problem is video, so the task reduces to analysing frames: locating the hand, isolating it from the background, describing its shape with numerical features and classifying that shape. Classical techniques such as colour-space conversion, thresholding, morphological filtering, contour analysis and moment invariants provide interpretable, lightweight descriptors, while modern learned detectors such as MediaPipe Hands provide precise landmark geometry. Using both gives a system that is accurate and also explainable.

Motivation. This project was chosen to apply the concepts studied in Image and Video Processing to a socially useful, end-to-end application: from raw camera frames, through segmentation and feature extraction, to a decision and an audible output. It also allowed a direct comparison between classical hand-crafted features and learned landmark features.

2. Problem Statement and Objectives

Problem statement. Hearing people who cannot read sign language cannot understand signers without an interpreter. The real-world problem addressed here is to recognise ASL fingerspelling from live webcam video and convert it into readable text and spoken words in real time, using only affordable hardware.

Objectives. The specific objectives of the project are:

- To capture live video from a webcam and detect the signer's hand in every frame.
- To implement a classical image-processing pipeline (HSV skin segmentation, morphological filtering and contour analysis) that extracts shape features from the hand region.
- To build a hybrid feature vector combining MediaPipe landmarks with the classical shape features, and to train a classifier on a self-recorded dataset covering 24 ASL letters (A–Y except X).
- To convert stable per-frame predictions into letters, words and sentences, and to speak them aloud using text-to-speech.
- To evaluate the contribution of each feature group (landmarks-only, classical-only, hybrid) and to document the limitations of the approach.

3. Scope of the Project

What the project covers

- Recognition of static ASL fingerspelling letters (24 classes: A–Y except X) performed with one hand in front of a webcam.
- A complete pipeline: data collection, feature extraction, model training, real-time recognition, word/sentence building and speech output.
- A standalone visualisation of the classical image-processing stages (skin mask, contour, convex hull, convexity defects).
- An ablation experiment comparing landmark-only, classical-only and hybrid feature sets.

What the project does not cover

- Continuous, sentence-level sign language translation and grammar; facial expressions and body posture.
- Two-handed signs and signs defined by motion. The letter J involves movement and is represented by a single hand pose in this static system; the letters X and Z were not recorded and are not recognised.
- Generalisation across many signers, skin tones and camera types (the dataset is from one signer).
- Other sign languages (for example Indian Sign Language), and mobile or embedded deployment.

Type of input. Live video from a laptop webcam, processed frame by frame; one hand in view, indoor ambient lighting, background containing ordinary room objects. **Application domain.** Assistive technology, education and human–computer interaction.

4. Dataset / Input Description

Source. No public dataset was used. The dataset was recorded by the author using the project's own data-collection tool (`data_collection.py`) and the laptop webcam. For each of 24 letters (A–Y except X) the author held the sign in front of the camera while the tool saved one feature vector per frame in which a hand was detected, varying hand angle and distance slightly to improve robustness. Raw frames are not stored; only the 76 numerical features per frame are saved to a CSV file.

Property	Value
Dataset name	Custom self-recorded ASL fingerspelling dataset
Classes	24 (letters A–Y except X; X and Z were not recorded)
Samples per class	123 (letter C) to 563 (letter U); mean ≈ 266
Total samples	6,396 feature vectors
Input resolution	[640 × 480 – confirm your webcam resolution]
Frame format	8-bit BGR from OpenCV; converted to RGB (MediaPipe) and HSV (skin segmentation)
Stored data format	CSV: label + 76 numeric features per row (63 landmark + 13 classical)
Video	Live webcam stream; each letter recorded for a few seconds per session
Signer / conditions	Single signer (the author); indoor fluorescent ceiling lighting, plain room background

Figure 1 shows the data-collection window while the letter A is being recorded. The coloured skeleton is the MediaPipe landmark overlay, and the status line shows the active label and recording state.

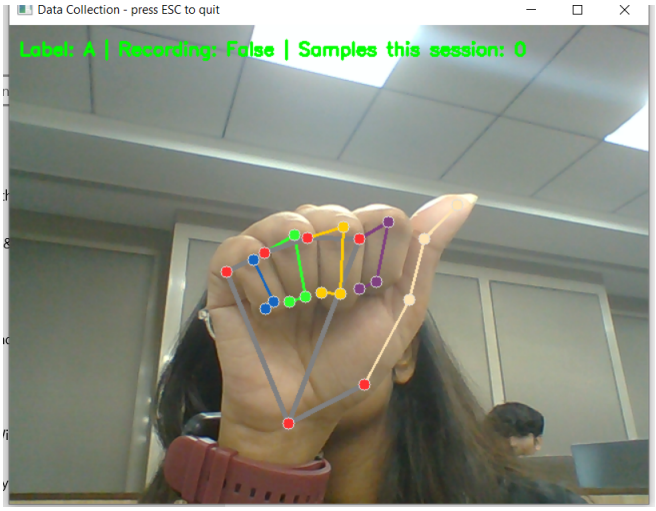


Figure 1: Data-collection window (label A selected), with MediaPipe hand landmark overlay.

5. Methodology / Proposed Approach

The system has an offline training phase and an online recognition phase that share the same feature extraction. Every frame passes through two parallel branches (Figure 2). The *landmark branch* detects the hand with MediaPipe, and normalises the 21 landmarks so that they do not depend on where the hand is or how large it appears. The *classical branch* crops the hand region (the landmark bounding box with a 25% margin), segments skin by HSV thresholding, cleans the mask by morphology, extracts the largest contour and computes 13 shape descriptors. The two feature groups are concatenated into a 76-dimensional vector and classified by a Random Forest. Finally, a temporal filter confirms letters and the text buffer is spoken.

Algorithm (pseudocode)

```
TRAINING (offline)
  for each recorded letter L:  record frames; save [L, features(frame)] to CSV
  split 80/20 (stratified); fit RandomForest(200 trees, max_depth=20); save model

RECOGNITION (online, per frame)
  frame   <- mirror(camera.read())
  hand    <- MediaPipeHands(RGB(frame))
  if no hand: reset stability counter; continue
  f_land  <- normalise(hand.landmarks)              # 63 values
  roi     <- crop(frame, bbox(hand) + 25% margin)
  mask    <- morph_open_close(blur(inRange(HSV(roi), skin_lo, skin_hi)))
  cnt     <- largest_contour(mask)
  f_cls   <- shape_descriptors(cnt) if cnt else zeros(13)   # 13 values
  (label, conf) <- RandomForest.predict_proba(concat(f_land, f_cls))
  if conf >= 0.60 and label stable for 10 frames and label != last accepted:
        word += label
  SPACE: commit word | b: backspace | c: clear | s: speak sentence (TTS)
```

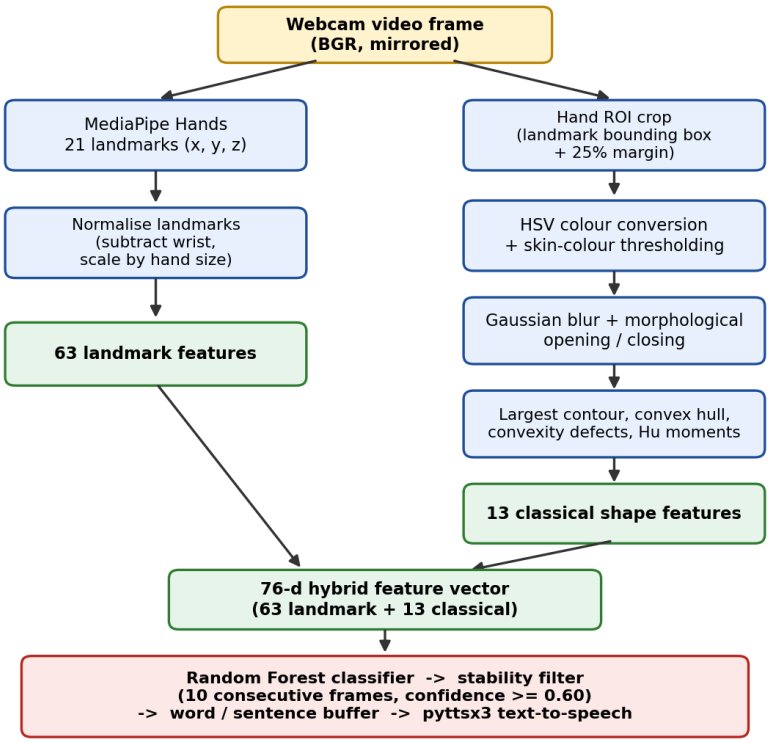


Figure 2: Block diagram of the proposed sign-to-speech pipeline.

6. Techniques / Algorithms Used

6.1 Image processing

The operations below follow the standard definitions in [10]; parameter values are those used in the code.

Technique	How it is used in this project
Colour-space conversion	BGR → RGB for MediaPipe; BGR → HSV for skin detection, because hue and saturation separate colour from brightness and are less sensitive to illumination than RGB.
Thresholding	`cv2.inRange` keeps pixels in the skin range (default H 0–25, S 30–150, V 60–255), producing a binary skin mask.
Filtering	5×5 Gaussian blur on the mask followed by re-thresholding at 127 to smooth jagged edges.
Morphological operations	5×5 elliptical kernel: opening (2 iterations) removes noise specks; closing (2 iterations) fills small holes in the hand blob.
Segmentation	Skin-colour segmentation restricted to the landmark-derived hand ROI, then the largest connected contour (area > 1500 px) is taken as the hand (border following, Suzuki–Abe [6]).
Shape / feature extraction	Convex hull; solidity = A / A_{hull}; extent = A / (w·h); aspect ratio w/h; circularity = 4πA / P^2; equivalent-diameter ratio; convexity-defect count (finger gaps, depth > 8% of the hand size); and 7 Hu moment invariants [5], log-scaled as $-\text{sign}(h)\cdot\log_{10}\lvert h\rvert$. Total: 13 features.
Landmark normalisation	Subtract the wrist landmark (translation invariance) and divide by the maximum distance from the wrist (scale invariance), giving 63 features.

6.2 Video processing

Technique	How it is used in this project
Frame acquisition	Frames are read continuously from the webcam with OpenCV and mirrored so the display behaves like a mirror.
Hand detection and tracking	MediaPipe Hands detects the hand and then tracks it across frames (minimum detection confidence 0.6, tracking confidence 0.5, one hand) [2][3].
Temporal stability filter	A letter is accepted only if the same label is predicted with confidence ≥ 0.60 for 10 consecutive frames; the signer must then change the sign before the same letter can be added again. This suppresses flicker and transition frames.
Not used	Background subtraction, optical flow and frame differencing were not required because the signs are static and the hand is localised by MediaPipe; they are listed as possible extensions for motion-based signs.

6.3 Classification and speech

A Random Forest classifier [4] with 200 trees and a maximum depth of 20 was trained on the 76-d vectors using scikit-learn [7]; it was chosen because it trains quickly on small tabular data, needs no GPU, and gives class probabilities used as the confidence score. The accepted text is converted to speech with the offline `pyttsx3` engine, which uses the operating system's voices [11].

7. Implementation

Language and environment. The system is written in Python 3.12 and runs on Windows 10. Development used the Command Prompt and a virtual environment. **Libraries.** OpenCV (image and video processing) [8], MediaPipe 0.10.14 (hand landmarks), scikit-learn 1.5.1 (Random Forest and metrics), NumPy 1.26.4, pandas 2.2.2, joblib 1.4.2 (model persistence) and pyttsx3 2.90 (text-to-speech). **Hardware.** Built-in laptop webcam; processor and memory: [fill in CPU model / RAM]. No GPU was used.

Module	Purpose
utils.py	MediaPipe detector, landmark normalisation, hybrid 76-d feature builder, landmark drawing
preprocessing.py	Classical pipeline: skin segmentation, morphology, contour, convexity defects, shape descriptors, skin-range calibration
classical_demo.py	Live visualisation of the classical stages (mask, contour, hull, defects) with palm calibration
data_collection.py	Records labelled feature vectors from the webcam into data/landmarks.csv
train_model.py	Stratified 80/20 split, Random Forest training, accuracy and per-class report, model saving
evaluate_ablation.py	Landmarks-only vs classical-only vs hybrid comparison (5-fold CV + held-out test)
realtime_recognition.py	Live recognition, stability filter, word/sentence buffer, speech output
main.py	Menu launcher for all of the above

Important implementation steps

- Create the virtual environment and install pinned library versions (MediaPipe 0.10.14 requires Python 3.9–3.12).
- Verify the classical pipeline with `classical_demo.py` and calibrate the skin range for the room lighting.
- Record the letters (24 in the final dataset) with `data_collection.py`, selecting a letter key, pressing SPACE to record and SPACE again to stop.
- Train the Random Forest with `train_model.py`; the model is saved to `model/sign_classifier.joblib`.
- Run `realtime_recognition.py`. Controls: SPACE commits a word, b deletes, c clears, s speaks the sentence, q quits.

Key parameters. Stability frames = 10; confidence threshold = 0.60; speech rate = 155 words per minute; test split = 20% (stratified, random seed 42); Random Forest: 200 trees, depth 20.

8. Results and Discussion

8.1 Classical image-processing pipeline

Figure 3 shows the pipeline on a well-segmented frame. The left panel is the camera frame with the contour (green), convex hull (blue) and convexity defects (red dots); the right panel is the binary skin mask after thresholding and morphological cleaning. The hand is cleanly isolated, and the red defect points fall in the gaps between the fingers and at the wrist. The measured descriptors for this frame were solidity = 0.60, extent = 0.39, aspect ratio = 0.69 and 5 defects. Solidity is low because the contour includes the wrist and forearm, which extend well below the palm.

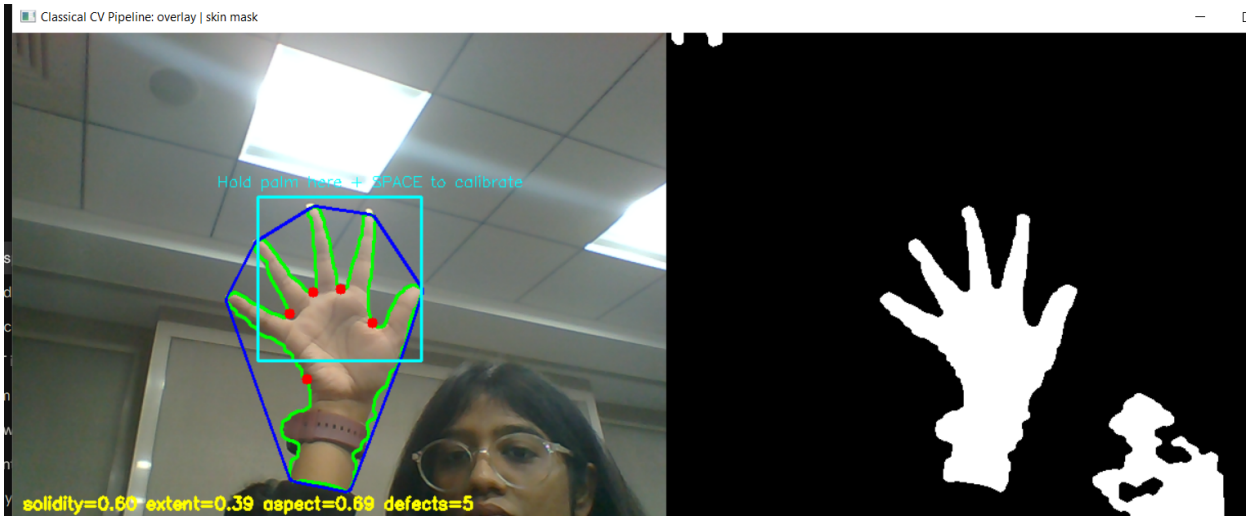


Figure 3: Classical pipeline output – overlay (left) and skin mask (right) for a well-segmented open palm.

Figure 4 shows a failure case. After skin-colour calibration the segmentation absorbed large parts of the ceiling and wall, so the contour merged with the background and the hull enclosed most of the frame. The descriptors (solidity 0.61, extent 0.42, 5 defects) were then not representative of the hand. The console log in Figure 5 explains why: while the calibration was repeated with the on-screen box partly covering non-skin pixels, the upper hue bound grew from about 41 to 100, which admits grey and bluish-white surfaces.

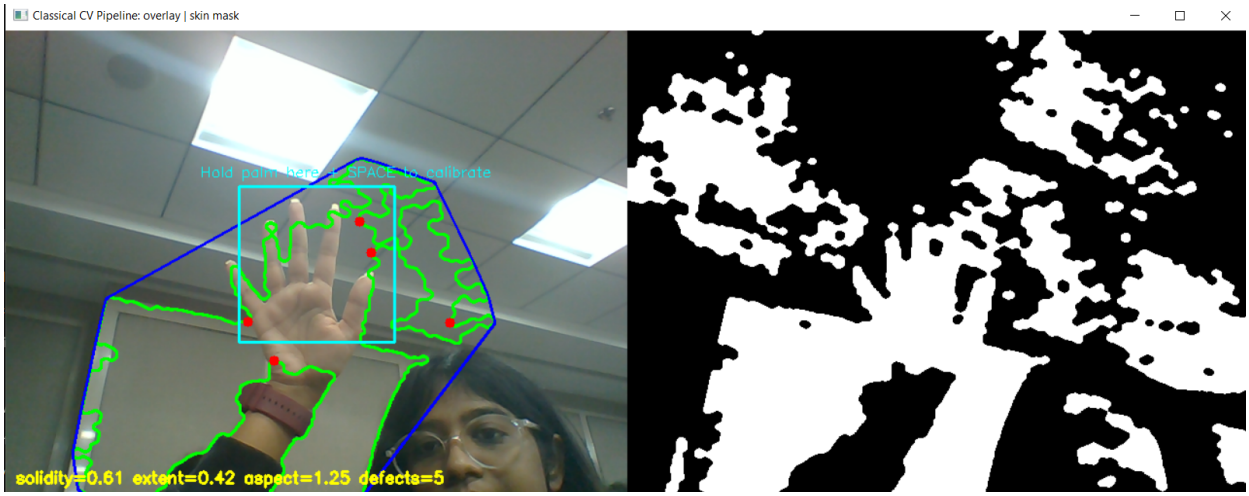


Figure 4: Failure case – over-wide calibrated HSV range makes the background part of the skin mask.

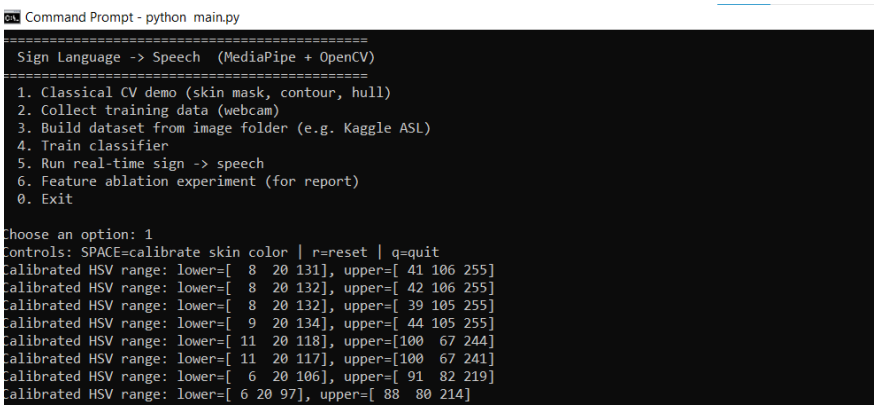


Figure 5: Console output of the calibration step, showing the HSV range drifting (upper hue 41 → 100) when non-skin pixels were sampled.

8.2 Model training and evaluation

The Random Forest was trained on the self-recorded dataset described in Section 4, with a stratified 80/20 split. Table 3 summarises the outcome.

Metric	Result
Total samples / classes	6,396 / 24
Training / test samples	5,116 / 1,280 (80/20 stratified split)
Test accuracy (hybrid features)	99.69 %
Macro-average precision / recall / F1	1.00 / 1.00 / 1.00
Lowest per-class scores	F1 = 0.99 for G, H, K, M, N, S, T and V; precision 0.98 for K, M, N; recall 0.98 for H, S, T, V; all other letters 1.00

Table 3: Training and evaluation summary.

The hybrid-feature Random Forest reached a test accuracy of 99.69%, which corresponds to about 4 misclassified frames out of 1,280 test samples. Figure 6 shows the number of samples recorded for each letter, and Figure 7 shows the console report produced by `train_model.py`.

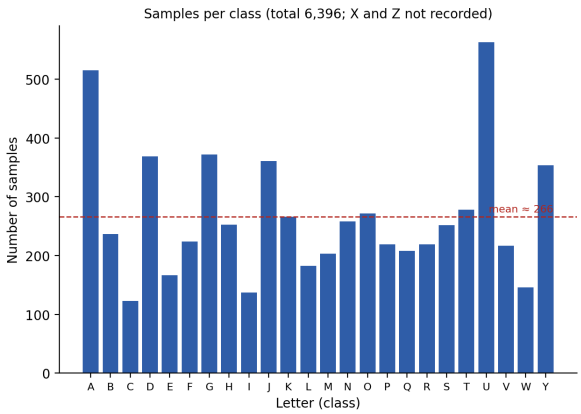


Figure 6: Number of recorded samples per letter.

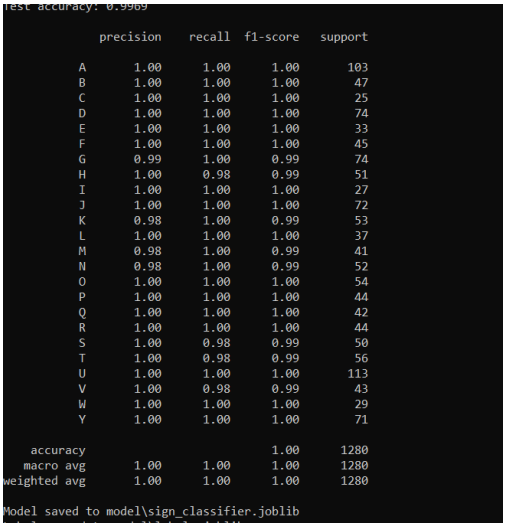


Figure 7: Console output of train_model.py (test accuracy and per-class report).

Interpreting the result. The accuracy is very high, but it should be read with care. The training and test samples were drawn at random from the same recording sessions of one signer, so near-identical

neighbouring frames can appear on both sides of the split. In the live application (Section 8.4), correct predictions were often made with a confidence of only 0.46–0.55, which suggests that performance on a fresh session, a different pose angle or a different signer is lower than the offline figure. A fairer evaluation would train on some recording sessions and test on entirely separate ones.

8.3 Feature ablation

To measure the value of the classical image-processing features, the same classifier was trained on three feature sets (`evaluate_ablation.py`).

Feature set	No. of features	5-fold CV accuracy	Test accuracy
Landmarks only (MediaPipe)	63	0.996 ± 0.002	0.996
Classical CV only (contour features)	13	0.883 ± 0.012	0.873
Hybrid (both)	76	0.996 ± 0.002	0.997

Table 4: Ablation of feature groups.

Landmarks alone and the hybrid set performed almost identically: test accuracy 0.996 versus 0.997, and the same cross-validation accuracy of 0.996 ± 0.002. The 0.001 difference corresponds to about one test sample and lies inside the cross-validation spread, so it should not be read as a genuine improvement. The 13 classical shape features alone reached 0.873 test accuracy (0.883 ± 0.012 in cross-validation). This is far above chance (about 4% for 24 classes) for features computed only from a colour-thresholded silhouette, but roughly 12 percentage points below the landmark features. On this dataset the landmarks therefore carry almost all of the discriminative information, and the classical features mainly act as an independent, interpretable cue. Their lower accuracy is consistent with the skin-segmentation weaknesses discussed in Section 9. As with Table 3, these figures come from random splits of one signer's recordings and are probably optimistic.

8.4 Real-time recognition

Figures 8 and 9 show the live application. The overlay displays the current prediction with its confidence, the word being built, the committed sentence and a status hint. In Figure 8 the signer shows a flat open palm; the classifier correctly predicts B with confidence 0.55, and the word buffer already contains A from the previous sign. Because 0.55 is below the 0.60 acceptance threshold, the letter is not yet added and the status reads "Hold sign steady". In Figure 9 the signer shows the curved C hand shape; the prediction C (confidence 0.46) is again correct, and the word buffer reads AB, showing that A and B were accepted earlier.

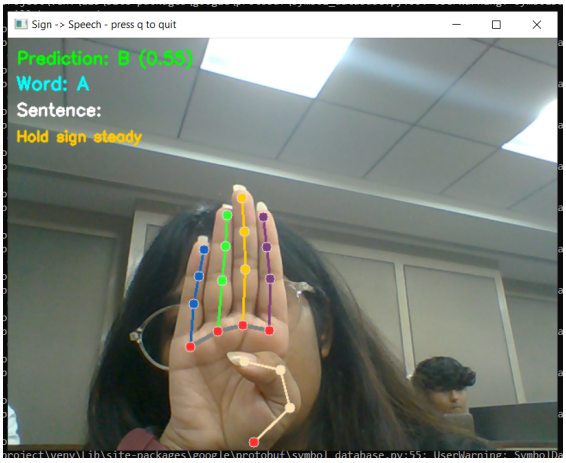


Figure 8: Sign B predicted with confidence 0.55; word buffer = "A".

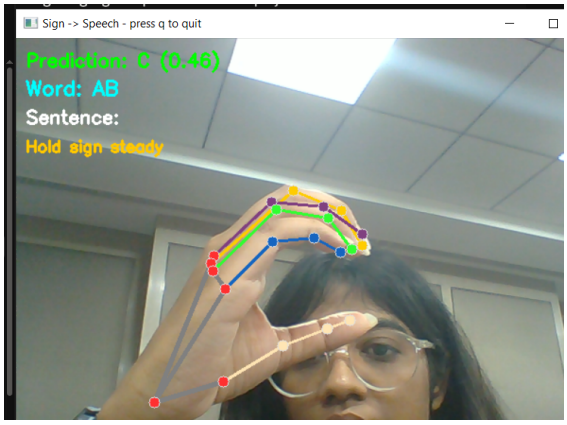


Figure 9: Sign C predicted with confidence 0.46; word buffer = "AB".

Figure	Signed letter	Predicted	Confidence	Accepted yet?	Word buffer
8	B	B	0.55	No (below 0.60)	A
9	C	C	0.46	No (below 0.60)	AB

Table 5: Predictions visible in the live screenshots.

Both displayed predictions were correct, but their confidence lay close to or below the acceptance threshold. This is expected behaviour of the design: low-confidence frames are ignored rather than written as wrong letters, at the price of requiring the signer to hold the sign steadily and clearly. The stability filter therefore trades speed for reliability.

8.5 Text and speech output

Spelling the word MAHIMA letter by letter produced the following console log. Each accepted letter extends the word, SPACE commits it to the sentence buffer, and pressing s sends the complete text to the speech engine, which then reads it aloud:

```
Accepted: M | Word: M
Accepted: A | Word: MA
Accepted: H | Word: MAH
Accepted: I | Word: MAHI
Accepted: M | Word: MAHIM
Accepted: A | Word: MAHIMA
Sentence buffer: [MAHIMA]
TEXT SENT TO SPEECH: [MAHIMA]
Speech completed.
```

Overall, the system met its functional objectives: a hand was detected in each frame, the classical and landmark features were combined, letters were recognised and stabilised, and the resulting text was spoken. Quantitative accuracy is reported in Tables 3 and 4.

9. Challenges and Limitations

- **Skin segmentation and lighting.** HSV thresholding is sensitive to illumination, skin tone and skin-coloured or light-grey backgrounds. Figure 4 is a direct failure case; when the mask fails the 13 classical features become unreliable (or zero), and the classifier depends on the landmarks alone.
- **Calibration instability.** Repeated calibration with the box not fully covering the palm produced an over-wide HSV range (Figure 5). The calibrated values are also used only in the demo; the recognition stage uses the default range.
- **Similar and low-confidence letters.** Letters with similar shapes (for example M/N/T or U/V) are harder to separate, and some correct predictions reached only 0.46–0.55 confidence (Figures 8 and 9), so the signer sometimes has to hold or adjust the sign.
- **Incomplete alphabet and motion-based letters.** Only 24 letters were recorded; X and Z are not recognised. J is defined by movement, so a static classifier can learn only a single pose for it, which makes it the least faithful class.
- **Dataset limitations.** All data came from one signer, one camera and one room. Consecutive frames of the same recording are almost identical, so a random train/test split places near-duplicate frames on both sides and the measured accuracy is likely to be higher than performance on a new signer or environment. Evaluating on a different person or session would give a fairer estimate. The classes are also imbalanced (123 samples for C against 563 for U) and no class weighting was used.
- **Computational limits.** Each frame requires MediaPipe inference, a skin segmentation, contour analysis and a Random Forest prediction on a CPU. This was acceptable for interactive use, but frame rate depends on the machine.
- **Text-to-speech.** With the Windows speech engine the first utterance was sometimes clipped (a word spoken as only its first syllable). The console log confirmed that the complete text was sent to the engine, and a separate test of the engine reproduced the effect, so the fault lay in speech playback and not in recognition. Speaking a short warm-up phrase before the first real sentence is the usual remedy.
- **Environment problems.** The MediaPipe version used does not support Python 3.13, so a Python 3.12 environment was required, and installation initially failed because of insufficient temporary disk space on the system drive.

10. Conclusion

What was implemented. A complete real-time system that recognises ASL fingerspelling from a webcam and converts it to text and speech, together with tools for data collection, training, ablation and visualisation of the image-processing stages.

Techniques used. HSV colour-space conversion, thresholding, Gaussian filtering, morphological opening and closing, contour extraction, convex hull and convexity-defect analysis, Hu moments, landmark normalisation, Random Forest classification and a temporal stability filter, combined with MediaPipe Hands for landmark detection.

What was achieved. The author-recorded 24-class dataset was classified with 99.69% test accuracy on a held-out split, and live demonstrations showed letters being recognised, collected into words and spoken aloud. The ablation study showed that the 13 classical features alone reach 87.3% accuracy, while adding them to the landmarks (99.7%) gave no measurable gain over landmarks alone (99.6%) on this dataset.

Learning. The project showed how classical image-processing operations and modern learned detectors complement each other, and also how strongly practical performance depends on lighting, calibration and data quality. Future work includes training on multiple signers, adaptive skin segmentation, recording sequences to handle motion-based signs (using LSTM or Transformer models on datasets such as WLASL [9]), supporting two hands, and moving to word-level translation.

References

[1] World Health Organization, “Deafness and hearing loss,” Fact Sheet. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

[2] F. Zhang et al., “MediaPipe Hands: On-device Real-time Hand Tracking,” arXiv:2006.10214, 2020.

[3] C. Lugaresi et al., “MediaPipe: A Framework for Building Perception Pipelines,” arXiv:1906.08172, 2019.

[4] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[5] M.-K. Hu, “Visual pattern recognition by moment invariants,” *IRE Transactions on Information Theory*, vol. 8, no. 2, pp. 179–187, 1962.

[6] S. Suzuki and K. Abe, “Topological structural analysis of digitized binary images by border following,” *Computer Vision, Graphics, and Image Processing*, vol. 30, no. 1, pp. 32–46, 1985.

[7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[8] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.

[9] D. Li, C. Rodriguez, X. Yu and H. Li, “Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison,” in *Proc. IEEE Winter Conf. on Applications of Computer Vision (WACV)*, 2020.

[10] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[11] pyttsx3: Offline text-to-speech library for Python. [Online]. Available: https://pypi.org/project/pyttsx3/